AI OPERATIONS — MLFLOW DEEP DIVE

Mastering MLflow for Experiment Tracking & Deployment

A self-contained, 2+ hour lecture + hands-on session: Tracking • Projects • Models • Registry • Serving

SESSION PLAN

Agenda — ~2 Hours 15 Minutes

Time	Block	Format
0:00 – 0:10	Why MLflow? Architecture recap	Lecture
0:10 – 0:40	Experiment Tracking deep dive (manual + autolog + search)	Lecture + code
0:40 – 0:55	Hands-on Exercise 1: instrument & compare runs	Lab
0:55 – 1:15	MLflow Projects: MLproject file, entry points, mlflow run	Lecture + code
1:15 – 1:25	Hands-on Exercise 2: package & run a project	Lab
1:25 – 1:45	MLflow Models: flavors, signatures, custom pyfunc	Lecture + code
1:45 – 1:55	Hands-on Exercise 3: log a model with a signature	Lab
1:55 – 2:10	Model Registry: register, stage/alias, promote	Lecture + code
2:10 – 2:25	Model Serving: local REST server, batch inference, Docker	Lecture + code

PREREQUISITES

Before You Start This Session

- Python 3.9+ environment with mlflow, scikit-learn, and pandas installed (pip install mlflow scikit-learn pandas).
- A terminal, a code editor, and a web browser for the MLflow UI.
- Familiarity with a basic scikit-learn or PyTorch training loop (covered in prior AIOps Module 1 / DL coursework).
- Recommended: complete this session's setup check — run `mlflow --version` and `python -c "import sklearn"` before class starts.
- This deck is self-contained: every concept is followed by a runnable code snippet and, where relevant, a hands-on exercise (marked in amber).

00

Why MLflow?

What this section covers

- The problem MLflow solves
- The four MLflow components
- Installing MLflow & starting a Tracking Server
- How the pieces fit together end-to-end

0.1 THE PROBLEM

Without vs. With MLflow

Without MLflow

- Hyperparameters scattered in notebook cells or print() statements.
- “Which run gave us 94% accuracy?” — nobody remembers.
- Models saved as loose .pkl files with no lineage back to the code that made them.
- Deploying a model means manually writing a Flask app from scratch.

With MLflow

- Every parameter, metric, and artifact logged automatically, searchable later.
- A UI and API to compare hundreds of runs side-by-side.
- A standard “Model” format that carries its own environment and metadata.
- One command turns a logged model into a REST API: mlflow models serve.

0.2 THE FOUR COMPONENTS

MLflow Is a Platform, Not Just a Logger

1

MLflow Tracking

Log and query parameters, metrics, artifacts, and source code across experiments and runs.

2

MLflow Projects

A standard, reusable format for packaging data science code so anyone can run it the same way.

3

MLflow Models

A standard format for packaging models so they can be used by many downstream tools and serving engines.

4

Model Registry

A centralized, versioned store with a governed lifecycle (stages/aliases) for collaborative model management.

Getting MLflow Running Locally

```
$ pip install mlflow scikit-learn pandas

# Start a local Tracking Server backed by SQLite,
# with artifacts stored on local disk:
$ mlflow server \
    --backend-store-uri sqlite:///mlflow.db \
    --default-artifact-root ./mlruns \
    --host 0.0.0.0 --port 5000

# In a second terminal, verify it's alive:
$ curl http://localhost:5000/health
```

Keep this server running in a dedicated terminal for the entire session — every exercise below logs to it.

How the Four Components Fit Together

1

1. Track

Your training script logs params/metrics/artifacts to the Tracking Server as it runs.

2

2. Package

An MLflow Project wraps that script so anyone can rerun it identically.

3

3. Save as a Model

The trained model is saved in MLflow's standard Model format.

4

4. Register

The model is registered and promoted through stages in the Model Registry.

5

5. Serve

The registered model is deployed as a REST API or batch job.

01

Experiment Tracking Deep Dive

What this section covers

- Core vocabulary recap
- Manual logging: params, metrics, artifacts, tags
- Autologging with one line of code
- Nested runs for hyperparameter sweeps
- Searching & querying runs programmatically

1.1 VOCABULARY RECAP

Six Terms, One Mental Model

- Experiment — a named collection of related runs.
- Run — a single execution of your code; the atomic unit of tracking.
- Parameter — static input configuration, logged once per run.
- Metric — a value that can be logged multiple times (e.g. once per training step/epoch).
- Artifact — any output file: model weights, plots, HTML reports, datasets.
- Tag — free-form key/value metadata, e.g. git commit hash, team name, dataset version.

The Core Tracking API

```
import mlflow
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("iris-classifier")

with mlflow.start_run(run_name="rf-baseline"):
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 5)

    model.fit(X_train, y_train)
    acc = model.score(X_test, y_test)

    mlflow.log_metric("accuracy", acc)
    mlflow.set_tag("team", "data-science")
    mlflow.log_artifact("feature_importance.png")
```

with mlflow.start_run(): auto-closes the run even on error — always prefer it over manual start_run()/end_run() calls.

Logging Metrics Over Steps, Dicts & Figures

```
with mlflow.start_run():
    for epoch in range(20):
        train_loss, val_acc = train_one_epoch(...)
        # step= makes this a TIME SERIES metric, plottable in the UI
        mlflow.log_metric("train_loss", train_loss, step=epoch)
        mlflow.log_metric("val_accuracy", val_acc, step=epoch)

    # Log several params/metrics at once
    mlflow.log_params({"lr": 0.001, "batch_size": 32})
    mlflow.log_metrics({"final_val_acc": val_acc, "epochs": 20})

    # Log a matplotlib figure directly (no need to save to disk first)
    mlflow.log_figure(fig, "confusion_matrix.png")

    # Log a Python dict as a JSON/YAML artifact
    mlflow.log_dict({"classes": ["setosa", "versicolor"]}, "labels.json")
```

One Line to Track (Almost) Everything

```
import mlflow
mlflow.autolog()   # auto-detects sklearn, PyTorch, Keras, XGBoost, LightGBM...

# From here, just train normally — no manual log_param/log_metric needed:
model = RandomForestClassifier(n_estimators=200, max_depth=8)
model.fit(X_train, y_train)

# MLflow automatically captures:
#   - all constructor hyperparameters
#   - training/validation metrics it can infer
#   - the fitted model itself, with a signature
#   - a text summary of the model

# Framework-specific variant (same idea):
mlflow.sklearn.autolog()
mlflow.pytorch.autolog()
```

Best practice: use autolog() for fast iteration, then switch to explicit manual logging for the final, reported run to control exactly what's captured.

Organizing a Hyperparameter Sweep

```
with mlflow.start_run(run_name="lr-sweep") as parent:
    for lr in [0.1, 0.01, 0.001]:
        with mlflow.start_run(run_name=f"lr-{lr}", nested=True):
            mlflow.log_param("lr", lr)
            acc = train_and_evaluate(lr)
            mlflow.log_metric("val_accuracy", acc)

# In the MLflow UI, child runs appear grouped and indented
# under the parent "lr-sweep" run — ideal for grid/random search.
```

nested=True is the key argument — without it, the second start_run() call raises an error because a run is already active.

Programmatic Access to Your Tracking History

```
import mlflow

# Pandas DataFrame of every run in an experiment
runs_df = mlflow.search_runs(experiment_names=["iris-classifier"])
print(runs_df[["run_id", "params.n_estimators", "metrics.accuracy"]])

# Filter with an SQL-like syntax, sorted by the metric that matters
best_runs = mlflow.search_runs(
    experiment_names=["iris-classifier"],
    filter_string="metrics.accuracy > 0.9 and params.max_depth = '5'",
    order_by=["metrics.accuracy DESC"],
)
best_run_id = best_runs.iloc[0].run_id
```

This is how CI/CD pipelines (Module 4) automatically pick the “best” run without a human opening the UI.

✎ HANDS-ON EXERCISE — 1

~10–15 MIN

Instrument & Compare Runs

1. Take the provided starter script (load_iris → train a classifier — no MLflow calls yet).
2. Add mlflow.set_experiment() and wrap training in with mlflow.start_run():.
3. Log at least 3 hyperparameters and 2 metrics manually.
4. Re-run the script 4 times varying one hyperparameter (e.g. n_estimators: 10, 50, 100, 300).
5. Add mlflow.autolog() to a 5th run and compare what got captured automatically vs. manually.
6. Open the MLflow UI, select all 5 runs, and use “Compare” to find the best one.

Deliverable: A screenshot of the 5-run comparison view, plus the run_id of the best run found via mlflow.search_runs().

02

MLflow Projects

What this section covers

- What a Project is and why it matters
- The MLproject file format
- Defining entry points & parameters
- Running a project locally, from Git, or in a container

Reusable, Reproducible Code Packaging

- A Project is simply a directory (or Git repo) containing an MLproject file that describes how to run the code.
- It answers the question: “How do I run this exact training job, with these exact dependencies, on someone else's machine?”
- Solves the same problem as Module 1's Conda/Mamba environment files — but standardized and runnable with a single CLI command.
- Projects can be run locally, directly from a GitHub URL, or inside a Docker container — same interface either way.

Declaring Entry Points & Parameters

```
name: iris-classifier

python_env: python_env.yaml   # or: conda_env: conda.yaml

entry_points:
  main:
    parameters:
      n_estimators: {type: int, default: 100}
      max_depth: {type: int, default: 5}
      data_path: {type: str, default: "data/iris.csv"}
    command: "python train.py --n_estimators {n_estimators} \
            --max_depth {max_depth} --data_path {data_path}"
  evaluate:
    parameters:
      model_uri: {type: str}
    command: "python evaluate.py --model_uri {model_uri}"
```

One MLproject file can define multiple entry points — e.g. one for training, one for evaluation — each independently runnable.

python_env.yaml (or conda.yaml)

```
# python_env.yaml — pins the exact interpreter + dependencies
python: "3.10.13"
build_dependencies:
  - pip
  - setuptools
dependencies:
  - scikit-learn==1.4.0
  - pandas==2.1.4
  - mlflow==2.11.0

# Conda alternative (conda.yaml):
# name: iris-env
# channels: [conda-forge]
# dependencies:
#   - python=3.10
#   - scikit-learn=1.4.0
#   - pip:
#     - mlflow==2.11.0
```

This is the Project-level analogue of Module 1's environment.yml — same idea, now tied directly to a runnable entry point.

The mlflow run Command

```
# Run the default ("main") entry point locally, overriding params
$ mlflow run . -P n_estimators=300 -P max_depth=8

# Run a specific entry point
$ mlflow run . -e evaluate -P model_uri=runs:/<run_id>/model

# Run directly from a Git repository — no local clone needed
$ mlflow run https://github.com/my-org/iris-project.git \
    -P n_estimators=200

# Force MLflow to build and run inside a fresh, isolated environment
$ mlflow run . --env-manager=conda
```

Every mlflow run automatically creates a new MLflow Tracking run too — Projects and Tracking are designed to work together, not separately.

✎ HANDS-ON EXERCISE — 2

~10–15 MIN

Package & Run an MLflow Project

1. Turn your Exercise 1 training script into train.py that accepts CLI arguments (argparse).
2. Write an MLproject file with a main entry point and at least 2 parameters.
3. Add a python_env.yaml pinning your key library versions.
4. Run it with: mlflow run . -P n_estimators=150 -P max_depth=6
5. Confirm a new run appears in the MLflow UI, logged automatically by the Project run.
6. Bonus: re-run with a different parameter value and compare both runs in the UI.

Deliverable: A working MLproject directory (MLproject + python_env.yaml + train.py) and a screenshot of the resulting run in the MLflow UI.

03

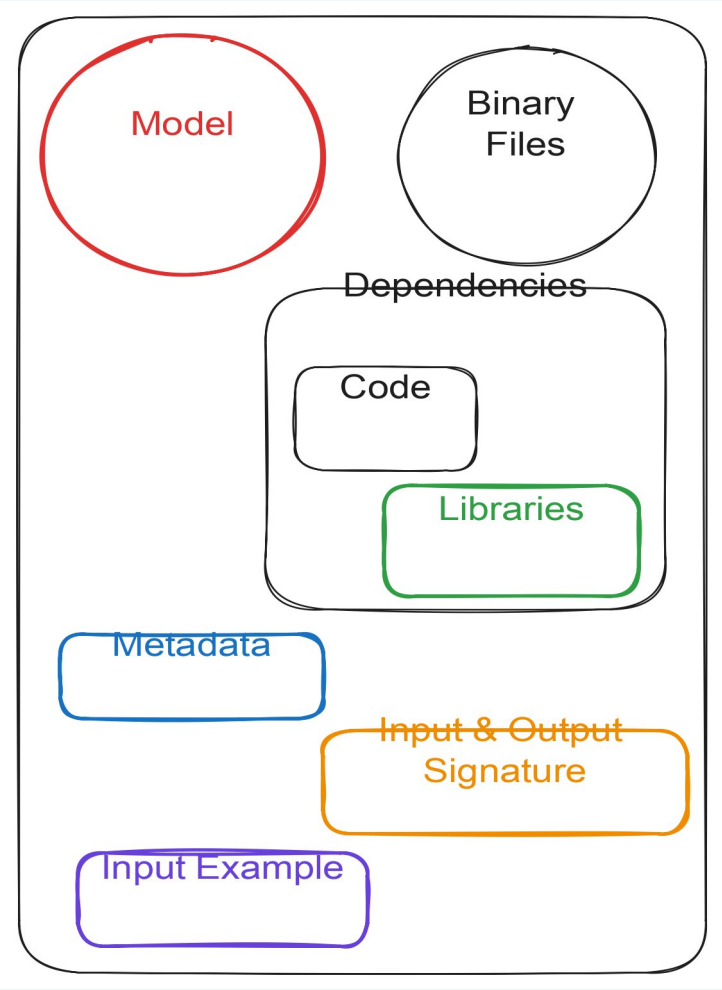
MLflow Models

What this section covers

- The “flavor” concept
- Anatomy of an MLmodel file
- Saving, logging & loading models
- Model signatures & input examples
- Custom models with pyfunc

One Format, Many Consumers

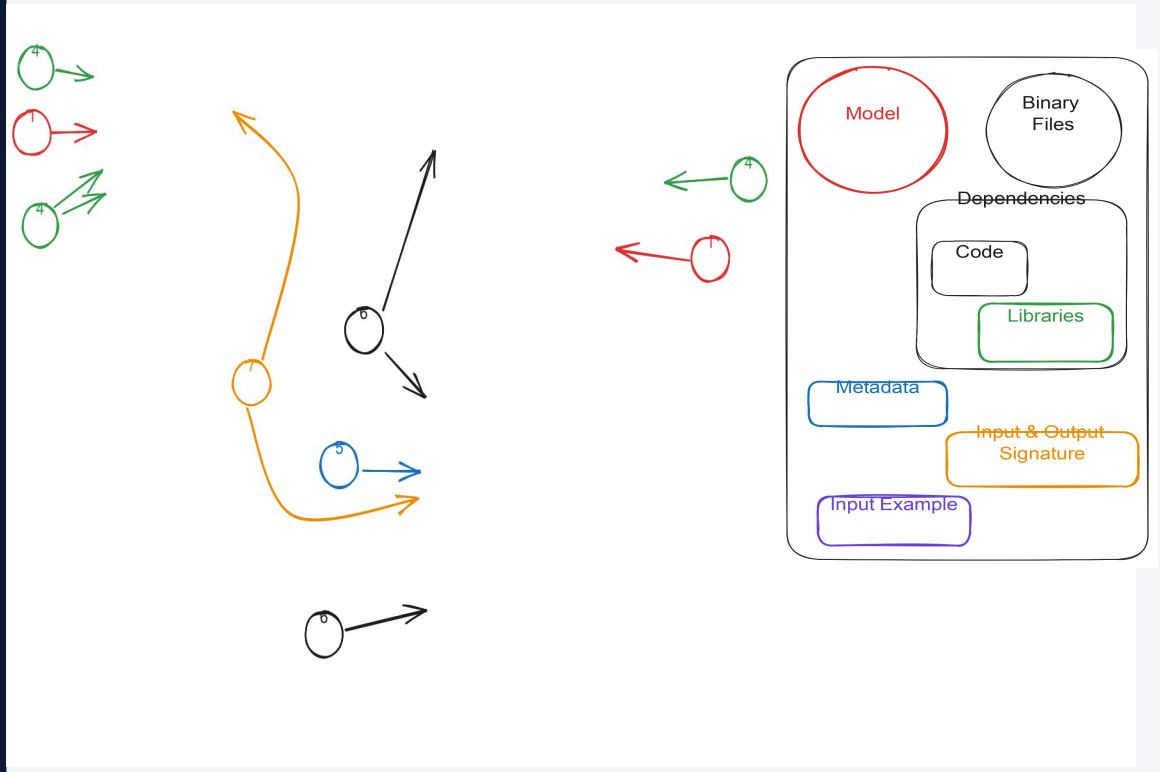
- An MLflow Model is a standard directory structure + metadata file (MLmodel) describing how to load and run a model.
- A single saved model can support multiple “flavors” simultaneously — e.g. a native sklearn flavor AND a generic python_function (pyfunc) flavor.
- The pyfunc flavor is the universal interface: any tool that speaks pyfunc (batch inference, REST serving, Spark UDFs) can load ANY MLflow Model, regardless of the framework it was trained in.
- This is what makes Module 5's LLM/GenAI serving and Module 4's deployment pipelines framework-agnostic.



What log_model() Actually Writes to Disk

```
my_model/
├── MLmodel              <- metadata: flavors, signature, run_id
├── model.pkl            <- the serialized sklearn estimator
├── conda.yaml            <- exact environment to reproduce inference
├── python_env.yaml
├── requirements.txt
└── input_example.json     <- optional sample input for testing

# MLmodel file excerpt:
flavors:
  sklearn:
    sklearn_version: 1.4.0
    pickled_model: model.pkl
  python_function:
    loader_module: mlflow.sklearn
signature:
  inputs: '[{"name": "sepal_length", "type": "double"}, ...]'
  outputs: '[{"type": "long"}]'
```



This bundle is exactly what travels through the Registry and into Serving — the environment ships with the model, not separately.

The Core Model API

```
import mlflow.sklearn

with mlflow.start_run():
    model.fit(X_train, y_train)
    # Logs the model AS AN ARTIFACT of the current run
    mlflow.sklearn.log_model(model, artifact_path="model")

# Save to a local path instead (no active run needed)
mlflow.sklearn.save_model(model, path="my_model")

# Load it back later — two ways:
loaded_native = mlflow.sklearn.load_model("runs:/<run_id>/model")
loaded_pyfunc = mlflow.pyfunc.load_model("runs:/<run_id>/model")

predictions = loaded_pyfunc.predict(X_test)
```

Prefer mlflow.pyfunc.load_model for anything downstream (serving, batch scoring) — it works identically no matter what flavor trained the model.

Making Models Self-Describing

```
from mlflow.models import infer_signature

predictions = model.predict(X_train)
signature = infer_signature(X_train, predictions)

with mlflow.start_run():
    mlflow.sklearn.log_model(
        model,
        artifact_path="model",
        signature=signature,
        input_example=X_train.iloc[:5],
    )

# Now MLflow validates incoming data shape/types at inference time,
# and auto-generates a sample request for the REST API docs.
```

A missing or wrong signature is the #1 cause of "works in Python, fails when served" bugs — always log one for anything headed to production.

Wrapping Arbitrary Logic as an MLflow Model

```
import mlflow.pyfunc

class ThresholdedClassifier(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        import joblib
        self.model = joblib.load(context.artifacts["model_path"])

    def predict(self, context, model_input):
        probs = self.model.predict_proba(model_input)[:, 1]
        return (probs > 0.7).astype(int)   # custom business threshold

with mlflow.start_run():
    mlflow.pyfunc.log_model(
        artifact_path="model",
        python_model=ThresholdedClassifier(),
        artifacts={"model_path": "model.pkl"},
    )
```

This is how RAG pipelines, ensembles, and pre/post-processing logic (Module 5) get packaged as a single deployable MLflow Model.

✎ HANDS-ON EXERCISE — 3

~10–15 MIN

Log a Model with a Signature

1. Using your Exercise 1 model, generate predictions on a small sample of training data.
2. Call infer_signature() to build a signature from that sample.
3. Re-log the model with mlflow.sklearn.log_model(..., signature=..., input_example=...).
4. Open the run in the MLflow UI and confirm the signature appears under the Artifacts tab (MLmodel file).
5. Load the model back with mlflow.pyfunc.load_model() in a fresh Python session and call .predict() on new data.

Deliverable: A run containing a model artifact with a valid signature, plus a short script proving pyfunc.load_model() + predict() works end-to-end.

04

The Model Registry

What this section covers

- Why a registry, beyond just logged runs
- Registering a model from a run
- Stages vs. the newer Aliases system
- Promoting, loading & auditing model versions

4.1 WHY A REGISTRY?

From “A Run's Artifact” to “A Governed Asset”

- A logged model lives inside one run — useful for experimentation, but not a stable name a downstream system can depend on.
- The Model Registry gives a model a persistent NAME (e.g. “iris-classifier”) with automatically incrementing VERSIONS (v1, v2, v3…).
- Each version keeps full lineage back to its source run — code, params, metrics, and training data references.
- Serving systems and CI/CD pipelines reference the registry by name + stage/alias (e.g. “iris-classifier@production”), not by a specific run_id — so promoting a new version requires no downstream code change.

From a Run to a Registered Version

```
import mlflow

# Option A: register directly while logging
with mlflow.start_run():
    mlflow.sklearn.log_model(
        model, artifact_path="model",
        registered_model_name="iris-classifier",
    )

# Option B: register an existing run's model after the fact
result = mlflow.register_model(
    model_uri="runs:/<run_id>/model",
    name="iris-classifier",
)
print(result.name, result.version)   # e.g. iris-classifier, 3
```

Registering the SAME name twice creates a new incrementing VERSION — nothing is ever silently overwritten.

Stages (Classic) vs. Aliases (Current Recommendation)

Concept	How It Works	Notes
Stages	Fixed set: None → Staging → Production → Archived, one active stage per version	Simple, but only ONE version can be “Production” at a time — being deprecated in newer MLflow
Aliases	Custom, mutable pointers you define, e.g. “@champion”, “@challenger”, “@shadow”	Multiple aliases can point to different versions simultaneously — enables A/B and shadow testing
Tags	Free-form key/value metadata on a version, e.g. validated_by, dataset_version	Used alongside either system for extra audit metadata

Stage Transitions & Alias Assignment

```
from mlflow import MlflowClient
client = MlflowClient()

# --- Classic stages API ---
client.transition_model_version_stage(
    name="iris-classifier", version=3, stage="Staging",
)
client.transition_model_version_stage(
    name="iris-classifier", version=3, stage="Production",
    archive_existing_versions=True,   # auto-archives the old Production version
)

# --- Newer aliases API (recommended going forward) ---
client.set_registered_model_alias(
    name="iris-classifier", alias="champion", version=3,
)
```

archive_existing_versions=True is what prevents two versions from both claiming to be “Production” at once.

Consuming the Registry Downstream

```
import mlflow

# Load by stage — always gets whatever version is CURRENTLY Production
model = mlflow.pyfunc.load_model("models:/iris-classifier/Production")

# Load by alias — the modern equivalent
model = mlflow.pyfunc.load_model("models:/iris-classifier@champion")

# Load an exact, pinned version (for audits / rollback)
model = mlflow.pyfunc.load_model("models:/iris-classifier/3")

# Full version history, for a compliance audit (recall Module 1, 3.4)
for v in client.search_model_versions("name='iris-classifier'"):
    print(v.version, v.current_stage, v.run_id, v.creation_timestamp)
```

Downstream serving code should almost always target a STAGE or ALIAS, never a hardcoded version number — that's what makes promotion a zero-code-change operation.

✎ HANDS-ON EXERCISE — 4

~10–15 MIN

Register & Promote a Model

1. Register the model from your Exercise 3 run under the name "my-classifier".
2. Register a second version by re-running training with different hyperparameters and registering again.
3. Transition version 1 to "Staging" and version 2 to "Production" (with archive_existing_versions=True).
4. Set an alias "@champion" pointing at whichever version has the best accuracy.
5. Write a short script that loads the model via models:/my-classifier/Production and via the @champion alias, and confirm both work.

Deliverable: A registered model with ≥ 2 versions, a visible stage/alias history in the UI, and a script proving stage- and alias-based loading both work.

05

Model Serving

What this section covers

- Local REST serving with one command
- Querying the inference API
- Batch & Spark UDF inference
- Building a serving Docker image

Zero-Code-Change Deployment

```
# Serve the current Production version of a registered model
$ mlflow models serve \
    -m "models:/iris-classifier/Production" \
    -p 5001 --env-manager=local

# Or serve directly from a run, without registering
$ mlflow models serve -m "runs:/<run_id>/model" -p 5001

# MLflow builds a REST API automatically:
#   POST /invocations   <- send data, get predictions
#   GET  /ping          <- health check
#   GET  /version       <- MLflow + model version info
```

No Flask app, no hand-written serialization code — the MLmodel signature is what makes this automatic.

Calling /invocations

```
$ curl -X POST http://localhost:5001/invocations \
    -H "Content-Type: application/json" \
    -d '{
          "dataframe_split": {
           "columns": ["sepal_length", "sepal_width",
                       "petal_length", "petal_width"],
           "data": [[5.1, 3.5, 1.4, 0.2]]
          }}'

# Equivalent Python client:
import requests
resp = requests.post(
    "http://localhost:5001/invocations",
    json={"dataframe_records": [{"sepal_length": 5.1, "sepal_width": 3.5,
                                 "petal_length": 1.4, "petal_width": 0.2}]},
)
print(resp.json())   # {"predictions": [0]}
```

If a request's schema doesn't match the logged signature, MLflow rejects it with a clear validation error — exactly the safety net Section 3.4 set up.

Scoring Without Standing Up a Server

```
import mlflow.pyfunc
import pandas as pd

# Simple batch scoring on a pandas DataFrame
model = mlflow.pyfunc.load_model("models:/iris-classifier/Production")
new_data = pd.read_csv("incoming_batch.csv")
predictions = model.predict(new_data)

# Distributed batch scoring as a Spark UDF (previewed for Module 2)
import mlflow.pyfunc
predict_udf = mlflow.pyfunc.spark_udf(
    spark, model_uri="models:/iris-classifier/Production",
)
scored_df = spark_dataframe.withColumn(
    "prediction", predict_udf(*feature_columns),
)
```

The exact same registered model URI works whether you're scoring 5 rows locally or 5 billion rows on a Spark cluster.

For Kubernetes / KServe Deployment (Module 3 Preview)

```
# Build a self-contained Docker image that serves the model,
# environment and all — no external MLflow server needed at runtime
$ mlflow models build-docker \
    -m "models:/iris-classifier/Production" \
    -n iris-classifier-image \
    --enable-mlserver

$ docker run -p 5001:8080 iris-classifier-image

# This image is what Module 3's Kubernetes/KServe deployment
# and Module 4's Canary/Blue-Green rollout consume directly.
```

--enable-mlserver swaps in the higher-throughput MLServer runtime — relevant again in Module 5 for GenAI-scale serving loads.

✎ HANDS-ON EXERCISE — 5

~10–15 MIN

Serve Locally & Query via curl

1. Start a local server: mlflow models serve -m "models:/my-classifier/Production" -p 5001 --env-manager=local.
2. In a second terminal, run the curl /invocations command against it with a real sample row.
3. Confirm the JSON response contains a sensible prediction.
4. Deliberately send a malformed request (wrong column name or missing field) and observe MLflow's validation error.
5. Bonus: write the equivalent Python requests call and run it from a script instead of curl.

Deliverable: A terminal transcript (or screenshot) showing one successful prediction and one rejected/invalid request, demonstrating signature validation in action.

WRAP-UP

MLflow Command & API Cheat Sheet

Task	Command / API
Start tracking server	mlflow server --backend-store-uri … --default-artifact-root …
Log a run manually	mlflow.start_run() / log_param / log_metric / log_artifact
Autolog everything	mlflow.autolog() or mlflow.sklearn.autolog()
Search runs	mlflow.search_runs(experiment_names=…, filter_string=…)
Run a Project	mlflow run . -P key=value
Log / load a Model	mlflow.<flavor>.log_model() / mlflow.pyfunc.load_model()
Register a Model	mlflow.register_model() or registered_model_name=…
Promote a version	client.transition_model_version_stage() / set_registered_model_alias()
Serve locally	mlflow models serve -m models:/name/Production -p 5001

KEY TAKEAWAY

One tool, one mental model, five stages:

Track → Package (Project) → Save (Model) → Register → Serve

Every exercise you completed today used the same run_id or model name from the previous step — that continuity IS the reproducibility protocol from Module 1, made concrete.

Next: apply this pipeline to a real model in Module 4 (CI/CD & Deployment Strategies).